

TDD-FDP-CCE

DESIGN SMART HOSPITAL BED MODEL USING ESP32 COMBINED IOT BLYNK

Project Name	Design smart hospital bed model using ESP32 combined IoT Blynk		
Student	[1] Võ Hoàng Khang	ID	21161245
Major	Electronics and Telecommunications Engineering	Supervisor	Assoc. Prof. Phan Van Ca

Contents

1. Introduction.....	5
2. Technical objectives and Specification.....	9
2.1. Customers Needs.....	9
2.2. Engineering Requirments.....	9
2.2.1 Funtions.....	9
2.2.2. Non-functions.....	9
2.2.3. Specification.....	9
3. Concept/Technology	10
3.1. Software overview:	10
3.2. Hardware overview	11
3.2.1. The Oled	11
3.2.2. MAX30100 Heart Rate and Blood Oxygen Sensor	11
3.2.3. HC-SR04 Ultrasonic Sensor	12
3.2.4. DFRobot Gravity: Offline Language Learning Voice Recognition Sensor - I2C & UART.....	13
3.2.5. SG90 180-degree servo motor	14
3.2.6. Module wifi ESP32	15
4. System Architecture.....	16
4.1. Introduction and Summary	16
4.1.1. General system requirements.....	16
4.1.2. Design options.....	16
4.2. System Block Diagram	17
4.3. Algorithm Flowchart.....	17
5. Detailed Design.....	19
5.1. Central processing block pin connection.....	19
5.2. Central processing block code	20
6. Final Product.....	32
6.1. Central processing block.....	32
6.2. Testing and evaluation:	33
6.3. Conclusion and development directions	33
7. Appendix.....	34
7.1. Division of Labor	34
7.2. Bill of Material	35
7.3. Gantt Chart	36
7.4. User manual	37

Revision History

Version	Date	Content of revision	Author	Supervisor
1.0	27-12-2024			
1.1				
1.2				
1.3				
1.4				
1.5				
2.0				

■ References

- [1] WHO, “Ageing and health”, who.int, 2024.
- [2] Nhung Nguyen, “Vietnam - Country Commercial Guide”, trade.gov, 2024.
- [3] Maria Clark, “ Medical Error Statistics: When Healthcare Can Kill You”, Etactics.com, 2020.
- [4] Ramakrishna Kakara, “ Nonfatal and Fatal Falls Among Adults Aged ≥ 65 Years — United States, 2020–2021”, cdc.gov, 2023.
- [5] Slow’IT, “ IoT in Medicine: How the Internet of Things Is Transforming the Medical Industry”, slowit.com, 2022.
- [6] Icdayroi, “Màn hình LCD Oled12864 1.3 inch”, icdayroi.com, 2024. [7] Kết nối ESP8266 và Arduino qua giao tiếp UART.
- [7] Nshop, “Cảm Biến Nhịp Tim Và Oxy Trong Máu MAX30100”, nshopvn.com, 2024.
- [8] Nshop, “Cảm Biến Siêu Âm HC-SR04”, nshopvn.com, 2024.
- [9] DFRobot, “Gravity: Offline Language Learning Voice Recognition Sensor for micro:bit / Arduino / ESP32 - I2C & UART”, dfrobot.com, 2024.
- [10] Nshop, “Động cơ servo SG90 180 độ”, nshopvn.com, 2024.
- [11] Nshop, “Kit RF thu phát wifi bluetooth esp32 Type C”, nshopvn.com, 2024.
- [12] Last Minute Engineer, “ESP32 Pinout Reference”, lastminuteengineers.com, 2024.

■ List of Tables & Figures

LIST OF IMAGES

Figure 1.1. Report of who has experienced with medical error
Figure 1.2. Age-adjusted percentage of adults aged ≥ 65 years reporting
Figure 1.3. Global healthcare IoT market size
Figure 3.1. Arduino IDE interface
Figure. 3.2. I2C LCD Oled12864 1.3 inch
Figure. 3.3. MAX30100 Sensor
Figure. 3.4. HC-SR04 Sensor
Figure. 3.5. DRRobot voice recognition
Figure. 3.6. Servo SG90
Figure. 3.7. Module ESP32
Figure. 3.8. Module ESP32 GPIO
Figure 4.1. System block diagram
Figure 4.2. Algorithmic flow chart
Figure 6.1. Central processing block
Figure 6.2. Final product

LIST OF TABLES

Table 5.1. Ultrasonic Sensor HC-SR04 pin connection diagram with ESP32 Node MCU
Table 5.2. Servo SG90 Leg pin connection diagram with ESP32 Node MCU
Table 5.3. Servo SG90 Head pin connection diagram with ESP32 Node MCU
Table 5.4. OLED pin connection diagram with ESP32 Node MCU
Table 5.5. DFRobot pin connection diagram with ESP32 Node MCU
Table 5.6. MAX30100 pin connection diagram with ESP32 Node MCU
Table 5.7. Buttons & Leds & Buzzer component pin connection diagram

1. Introduction

❖ Problem statement:

Currently, the demand for healthcare for the elderly and patients is increasing significantly. According to WHO projections, the global population of people over 60 will rise to 1.4 billion by 2030, reaching 2.1 billion by 2050. ¹In Vietnam, hospitals currently receive 60% of the nation's patients and consistently operate at over 200% capacity. ²

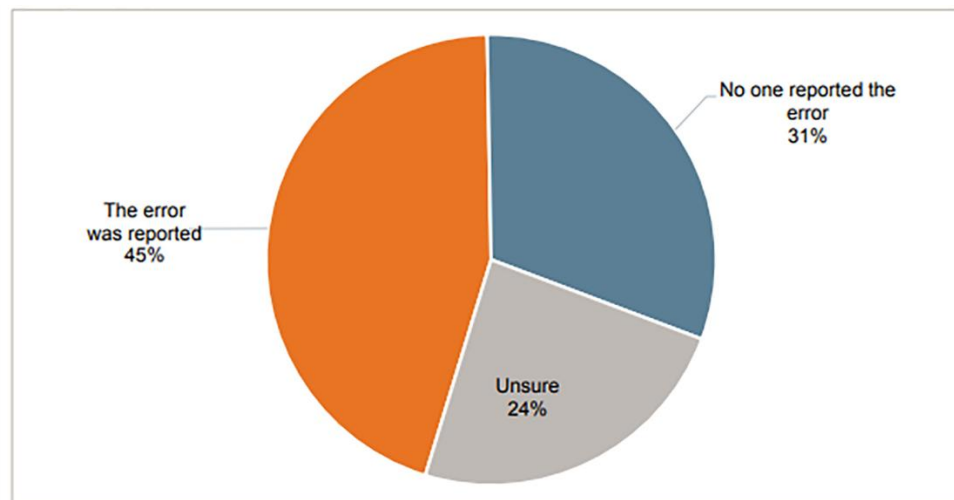


Figure 1.1. Report of who has experienced with medical error³

In addition, a serious issue in the current healthcare system is medical errors, which frequently occur but are often underreported. According to a 2017 survey by NORC at the University of Chicago, only 45% of medical errors in hospitals are reported, while 55% of cases from patients are either uncertain or unreported. This reveals a lack of monitoring and response to errors, which can have serious consequences for patient health.

Key causes include inadequate knowledge transfer within organizations, inaccurate information during treatment, patient-related issues, and improper workflows or staffing allocations. Technical and policy-related issues in many hospitals also contribute to this problem. Specifically, the lack of automatic monitoring and alert systems for patient health conditions (such as heart rate, body temperature, and blood oxygen levels) increases the risk of undetected abnormal signs, leading to serious

¹WHO, "Ageing and health", who.int, 2024.

² Nhung Nguyen, "Vietnam - Country Commercial Guide", trade.gov, 2024.

³ Maria Clark, "Medical Error Statistics: When Healthcare Can Kill You", Etactics.com, 2020.

consequences. For example, incidents like falls from beds among elderly patients remain a significant concern.

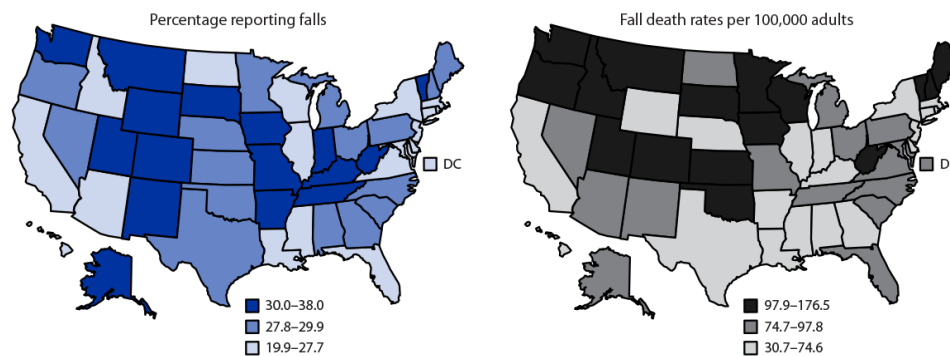


Figure 1.2. Age-adjusted percentage of adults aged ≥ 65 years reporting⁴

According to data from the Behavioral Risk Factor Surveillance System (BRFSS) in 2020 and the National Vital Statistics System (NVSS) in 2021 in the United States, the rate of older adults (aged 65 and older) who reported falling one or more times in the past year is alarmingly high. The mortality rate due to unintentional falls, including falls from beds, has significantly increased in this age group. Falls are a leading cause of serious injury and death among older adults, particularly as their physical health deteriorates over time. Neglect or lack of preventive measures in care and monitoring can lead to serious accidents, increasing the mortality rate due to falls among the elderly.

These statistics highlight the growing demand for healthcare for the elderly and patients, while medical errors continue to occur without being fully reported. Timely health monitoring is essential to ensure their safety, especially in preventing dangerous incidents like falls from beds, which are a primary cause of injuries and deaths among the elderly. Therefore, investment in smart monitoring systems not only improves the quality of healthcare but also provides convenience and peace of mind for patients and their families, reducing risks and enhancing medical experiences.

In the context of modern technology, applying the Internet of Things (IoT) in healthcare has become easier than ever. Connected medical devices such as smartwatches, health sensors, and monitoring devices enable users to track their health status automatically and continuously. By gathering data like heart rate, blood pressure, and physical activity, healthcare providers can remotely monitor patients' health and

⁴ Ramakrishna Kakara, “Nonfatal and Fatal Falls Among Adults Aged ≥ 65 Years — United States, 2020–2021”, cdc.gov, 2023.

deliver personalized care. This not only improves disease management but also reduces the burden on healthcare systems and limits unnecessary hospital admissions.

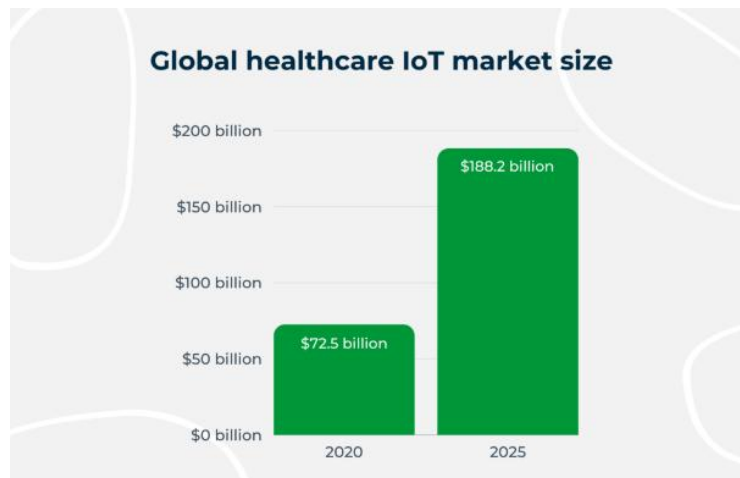


Figure 1.3. Global healthcare IoT market size⁵

Numerous forecasts and market studies confirm that this trend is where the industry is heading. For instance, a study by Markets and Markets predicts that the global IoT healthcare market will grow from \$72.5 billion in 2020 to \$188.2 billion by 2025. To meet these demands, this report presents a project **“DESIGN SMART HOSPITAL BED MODEL USING ESP32 COMBINED IOT BLYNK FOR REMOTE MONITORING”**. This smart bed model not only provides adjustable positioning functions but also incorporates smart sensors to continuously monitor the health status and conditions of patients.

❖ Target

The main objectives of this project include:

- Research theoretical foundations for hardware components such as the DFRobot Offline Voice Recognition Sensor, MAX30100 heart rate and SpO2 sensor, OLED display, ESP32 module, and other related devices.
- Calculate and design the schematic for hardware based on the studied components.
- Program software and establish a connection with the Blynk server via Wi-Fi, applying the acquired knowledge.
- Evaluate challenges and errors encountered during the implementation process, propose solutions, and suggest ways to address these issues.

❖ Content

The project aims to develop a smart hospital bed model to enhance comfort, safety for patients, and support healthcare. However, due to financial and technical constraints, the project will be researched within the following scope:

- **Model Design:** Focus on developing the model, without implementing it at a full-scale level.

⁵ Slow'IT, “IoT in Medicine: How the Internet of Things Is Transforming the Medical Industry”, slowit.com, 2022.

- **Information Security:** The project does not guarantee high levels of security for user data at this stage.

Despite these limitations, the project will include essential features to meet the following needs:

- **Position Adjustment:** The bed can raise and lower the head section, providing comfort for the patient.

- **Alarm System:** Alerts when a patient falls out of bed, ensuring timely safety measures.

- **Health Monitoring:** Continuously measures heart rate and blood oxygen levels (SpO2).

- **Smart Voice Control:** The patient can control the bed voice commands.

- **Display Screen:** Provides information about the patient's health status.

- **Remote Monitoring:** Health data is updated to a server and monitored remotely via phone or computer, with timely notifications.

2. Technical objectives and Specification

2.1. Customers Needs

- The system continuously monitors the patient's vital signs to ensure real-time health tracking.
- It provides alerts when the patient's condition becomes unstable or shows signs of health issues.
- The patient can call for assistance or adjust the bed's position to enhance comfort and support.

2.2. Engineering Requirments

2.2.1 Funtions

- The system must be capable of reading heart rate and SpO2 values from sensors and displaying these values directly at the bedside for real-time health monitoring.
- The bed must support adjustable positioning, allowing the head and foot sections to be raised or lowered to enhance the patient's comfort.
- Sensors must be included to detect if the patient is on the bed, and the system should activate an indicator light or buzzer to alert caregivers if the patient falls or leaves the bed unexpectedly.
- The system must allow patients to call for assistance using either a physical button or voice commands.
- The system must be integrated with the Blynk platform to enable remote monitoring of health data and to send emergency alerts when critical health issues are detected.

2.2.2. Non-functions

- The system must include a waterproof casing to protect the circuits from environmental damage.
- The design should be compact and aesthetically pleasing for easy installation on the hospital bed.
- The power supply must be stable and compatible with commonly available sources like adapters or batteries.
- The system must operate reliably in varying hospital environments and allow easy maintenance or part replacement.

2.2.3. Specification

- Input voltage: 5V.
- Circuit dimensions: 5 x 3 cm.
- Sensor accuracy: $\pm 5\%$.
- Patient detection range: 50-100 cm.
- OLED screen displays Heart Rate, SpO2, and bed status.
- Buzzer volume: 70 dB, with a response delay of under 5 seconds for alerts.
- The system supports voice commands and button controls for bed adjustments and assistance calls.
- Blynk integration updates health data remotely and sends emergency notifications instantly.

3. Concept/Technology

To design hardware and software, the team learned the following knowledge:

3.1. Software overview:

Overview of the Arduino IDE

The Arduino IDE is a software platform used to write, compile, and upload code to Arduino boards. It supports C/C++ programming languages and provides an easy-to-use interface with features like a code editor, compiler, and Serial Monitor for debugging. The IDE is compatible with various Arduino boards and comes with built-in libraries to simplify hardware integration, making it ideal for prototyping and embedded system projects. It's available for Windows, macOS, and Linux, and is open-source, allowing for customization and extension.

Here is some general information about the Arduino IDE software:

- Open-source and free: The Arduino IDE is an open-source software that is free for users.
- Supports C and C++ programming languages: The IDE uses C/C++ as the primary programming languages.
- User-friendly interface: It includes a code editor, a compiler for error checking, and a Serial Monitor for communication and debugging with the board.
- Compatible with multiple operating systems: It supports Windows, macOS, and Linux.
- Supports various Arduino boards: The IDE is compatible with a wide range of Arduino boards.
- Built-in libraries: The Arduino IDE includes libraries to facilitate communication with external components like sensors, motors, and displays.

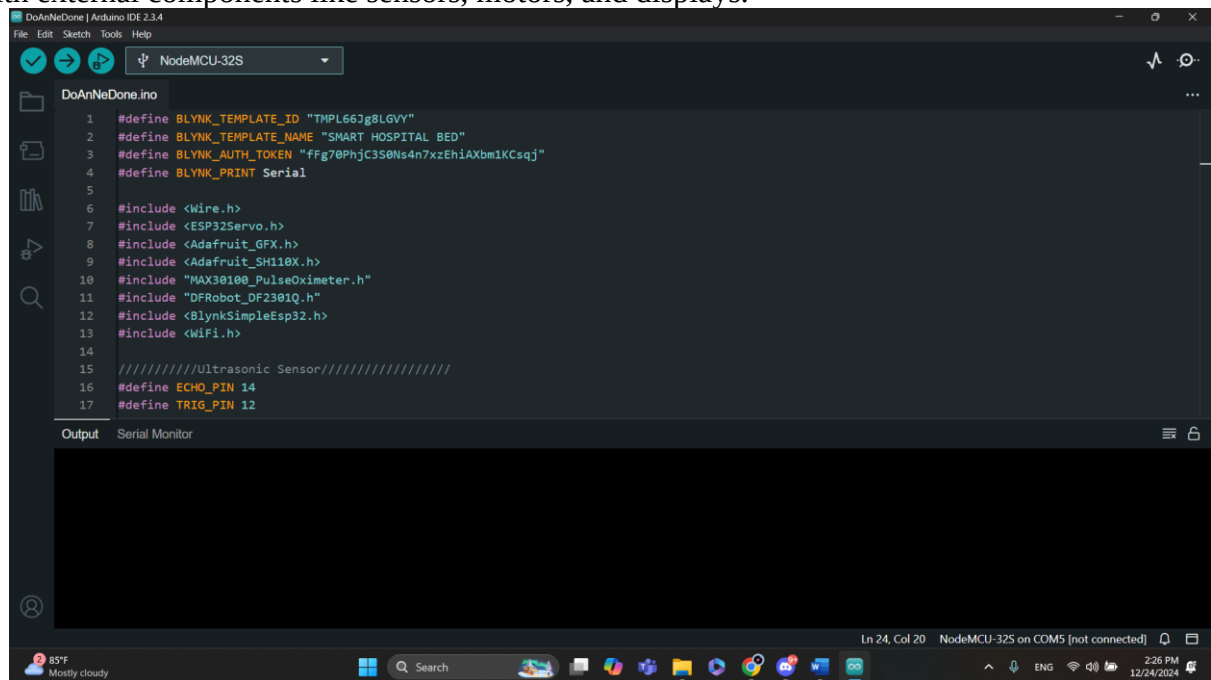


Figure 3.1. Arduino IDE interface

3.2. Hardware overview

3.2.1. The Oled

Oled LCD screen 1.3 inch with I2C interface for beautiful, luxurious display, clear daytime clarity and maximum energy saving with appropriate level, the screen uses I2C communication next for quality. Stable transmission quality and extremely easy communication with only 2-pin GPIO.⁶

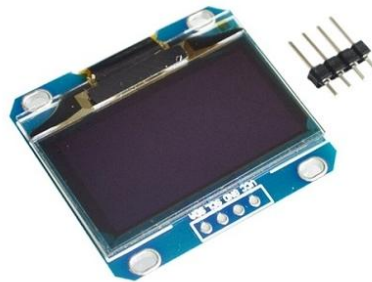


Figure. 3.2. I2C LCD Oled12864 1.3 inch

Pin:

- VCC: 3.3~5.5VDC
- GND: 0VDC
- SCL: Clock
- SDA: input data

Parameter:

- Supplied Voltage: 3.3~5.5VDC
- Power Consumption: 0.03w
- Display angle: greater than 160 degrees
- Number of pixel displayed: 128x64 pixels
- Communication standard: I2C
- Driver: SSD1106

3.2.2. MAX30100 Heart Rate and Blood Oxygen Sensor

The MAX30100 sensor is a device that measures heart rate and blood oxygen levels (SpO2) using advanced optical technology. It offers high accuracy and durability, making it ideal for biomedical applications. The MAX30100 is easy to use thanks to its I2C communication and Arduino support libraries, enabling quick integration into health monitoring projects.⁷

⁶ Icdayroi, "Màn hình LCD Oled12864 1.3 inch", icdayroi.com, 2024.

⁷ Nshop, "Cảm Biến Nhịp Tim Và Oxy Trong Máu MAX30100", nshopvn.com, 2024.



Figure. 3.3. MAX30100 Sensor

Pin:

- **VIN:** Power supply pin.
- **GND:** Connect to Supply ground.
- **SCL:** Serial Clock pin for I2C Serial Communication.
- **SDA:** Serial Data pin for I2C Serial Communication.
- **INT:** Active Low Interrupt pin.
- **IRD:** IR LED Cathode and LED Driver Connection pin.
- **RD:** Red LED Cathode and LED Driver Connection pin.

Parameter:

- **Operating Voltage:** 1.8~5.5VDC
- Compact, ultra-low power consumption, suitable for small devices and wearable devices
- **Communication:** I2C, TTL signal level
- **Dimensions:** 1.9 cm x 1.4 cm x 0.3 cm

3.2.3. HC-SR04 Ultrasonic Sensor

The SR04 ultrasonic sensor operates on the principle of ultrasonic wave reflection. It consists of two modules: one that emits ultrasonic waves and another that receives the reflected waves. The sensor emits a 40 kHz ultrasonic wave, and if there is an obstacle in its path, the wave reflects back to the receiving module. By measuring the time taken from emission to reception, the sensor calculates the distance between itself and the obstacle.⁸



Figure. 3.4. HC-SR04 Sensor

⁸ Nshop, “Cảm Biến Siêu Âm HC-SR04”, nshopvn.com, 2024.

Pin:

- VCC: Power supply.
- Trig: Trigger pin for emission.
- Echo: Echo pin for receiving feedback.
- GND: Ground connection.

Parameter:

- Voltage: 5V DC
- Operating Current: < 2mA
- High Level: 5V
- Low Level: 0V
- Max Angle: 15 degrees
- Distance: 2cm – 450cm (4.5m)
- Accuracy: 3mm

3.2.4. DFRobot Gravity: Offline Language Learning Voice Recognition Sensor - I2C & UART

The Offline Speech Recognition Module is ideal for human-computer interaction, voice assistants, and smart home projects. It supports Arduino, micro, and ESP32 with plug-and-play functionality via I2C and UART. With 121 built-in commands and 17 customizable ones, it includes an onboard speaker and microphone for real-time voice feedback, making it an efficient, easy-to-use solution.⁹

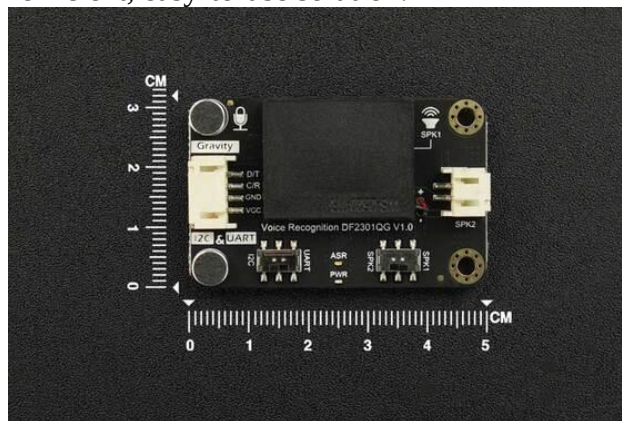


Figure. 3.5. DFRobot voice recognition

Pin:

- VIN: Power supply.
- GND: Connect to Supply ground.
- C/R: Command/Recognition Pin.
- D/T: Data/Transmission Pin.
- SPK2: Connecting to a speaker.

Parameter:

- Operating Voltage: 3.3 - 5V
- Maximum Operating Current: ≤ 370 mA (5V)
- Communication: I2C/UART
- I2C Address: 0x64
- Fixed Command: 121

⁹ DFRobot, "Gravity: Offline Language Learning Voice Recognition Sensor for micro:bit / Arduino / ESP32 - I2C & UART", dfrobot.com, 2024.

- Fixed Wake-up Command: 1
- Custom Command: 17
- Learning Activation Command: 1
- Onboard Microphone Sensitivity: -28db
- Module Size: 49×32 mm
- Operating Temperature: 0-70°C

3.2.5. SG90 180-degree servo motor

The SG90 180-degree servo motor offers a fast response time and is equipped with plastic gears, which makes it important to avoid heavy loads that could damage them. The RC Servo 9G comes with a built-in driver, allowing for easy control of the rotation angle through PWM (Pulse Width Modulation) signal.¹⁰



Figure. 3.6. Servo SG90

Pin:

- Red wire: Power supply.
- Brown wire: Connect to ground.
- Orange wire: signal.

Parameter:

- Operating voltage: 4.8-5VDC.
- Speed: 0.12 sec/60 deg (4.8VDC).
- Torque: 1.6 Kg.cm.
- Dimensions: 21x12x22mm.
- Weight: 9g.
- Control method: PWM.
- Pulse width: 0.5ms ~ 2.5ms corresponding to 0-180 degrees.
- Frequency: 50Hz, cycle: 20ms.

¹⁰ Nshop, “Động cơ servo SG90 180 độ”, nshopvn.com, 2024.

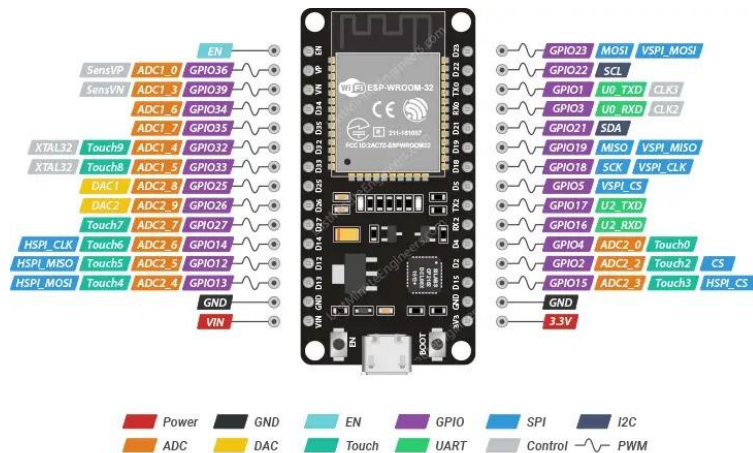
3.2.6. Module wifi ESP32

The ESP32 RF transceiver kit combines Wi-Fi and Bluetooth functionality with an integrated antenna, offering low power consumption, stable performance, and good interference resistance. This cost-effective solution is ideal for projects utilizing 2.4 GHz Wi-Fi and low-energy Bluetooth, built on TSMC's 40nm technology.¹¹



Figure. 3.7. Module ESP32

Pin:



ESP32 Dev. Board Pinout

Last Minute ENGINEERS.com

Figure. 3.8. Module ESP32 GPIO¹²

Parameter:

- Model: ESP32 38-pin
- Power supply (USB): 5V DC
- Input/Output voltage: 3.3V DC
- Power consumption: 5μA in sleep mode
- Performance: Up to 600 DMIPS
- Frequency: Up to 240MHz

¹¹ Nshop, "Kit RF thu phát wifi bluetooth esp32 Type C", nshopvn.com, 2024.

¹² Last Minute Engineer, "ESP32 Pinout Reference", lastminuteengineers.com, 2024.

4. System Architecture

After I has learned some necessary related concepts, the team proceeds to design and build the hardware and software.

4.1. Introduction and Summary

4.1.1. General system requirements

The system must meet the following requirements to ensure reliable operation and safety:

The system uses sensors to monitor heart rate, SpO₂, and bed occupancy.

- Health data must be sent to a remote server or mobile device for real-time monitoring.
- The bed can be adjusted for comfort using motors and controlled through physical buttons or voice commands.
- The system must activate warning lights and a buzzer if the patient falls or leaves the bed.
- The system must support wireless communication (Wi-Fi) for remote control and alerts.

4.1.2. Design options

With the requirements of the smart hospital bed system, the team has proposed the following design options for the hardware:

- Processing Block: The processing unit will use the ESP32 module, which provides stable 3.3V and 5V power, ample memory for the project, and supports Wi-Fi and Bluetooth for remote communication.
- Sensor Block: The MAX30100 sensor is chosen for its stable heart rate and SpO₂ measurements within an affordable price range, ensuring reliable health monitoring. The HC-SR04 ultrasonic sensor is selected for bed occupancy detection. It offers stable performance, a range greater than 100cm, and is safe for patients as it uses non-intrusive ultrasonic waves.
- Display Block: A 1.3-inch OLED screen with I2C communication will be used for displaying health metrics (heart rate, SpO₂), and it is energy-efficient for continuous use.
- Motor Control: For adjusting the bed's position, the SG90 Servo motor is chosen, as it is cost-effective and sufficient for the model's requirements.
- Voice Control: The DFRobot voice recognition module will be used for voice command input. Despite its higher cost, it is easy to program and comes with integrated microphone and speaker features.
- Power Block: The system will be powered via USB, with future plans to develop a separate power source to accommodate all components more efficiently.
- Remote Monitoring (Blynk Integration): The Blynk app will be used for remote monitoring and notifications. The ESP32 module will connect to a Wi-Fi network, allowing real-time health data (heart rate, SpO₂) to be sent to the Blynk cloud server. Users will be able to receive notifications or alerts if any abnormal readings are detected. The app will also provide the ability to remotely adjust the bed's position and check the status of sensors, providing easy access and control from a mobile device.

4.2. System Block Diagram

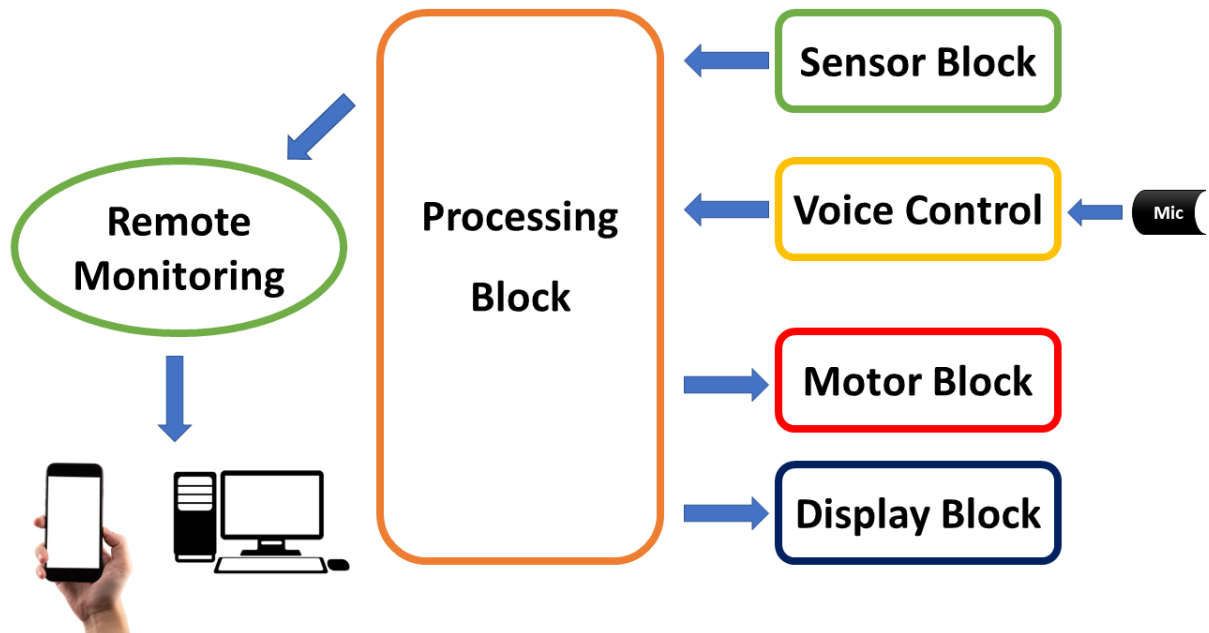


Figure 4.1. System block diagram

- Functions of blocks:

- **Processing Block:** Processes data, powers the system, and enables remote communication.

- **Sensor Block:** Measures health parameters and detects bed occupancy.

- **Display Block:** Displays health metrics and system information.

- **Motor Control Block:** Adjusts the bed's position.

- **Voice Control Block:** Accepts and processes voice commands.

- **Remote Monitoring:** Enables real-time monitoring, notifications, and remote control via a mobile app.

General operating principle: The system measures the patient's Heart Rate and SpO2 levels, displaying the data on both an OLED screen and the Blynk app. If the readings exceed predefined thresholds, an alert is triggered via a buzzer and a notification is sent to the Blynk app. Additionally, the system allows users to press a button or use voice commands to call for assistance or adjust the bed's position.

4.3. Algorithm Flowchart

The software comprises two main components: the `setup()` function for initialization and the `loop()` function for continuous operation. Each plays a vital role in ensuring the system functions smoothly, enabling real-time patient monitoring and interaction with caregivers.

The `setup()` function initializes all necessary hardware and software components. It begins by declaring and configuring the GPIO pins for sensors, LEDs, and other peripherals. The MAX30100 sensor is initialized to measure heart rate and SpO2 levels, while the OLED display is set up to show real-time health data. The system also connects

to Wi-Fi and initializes the Blynk platform to enable remote monitoring and notifications. This ensures that all components are fully operational and ready for the main loop's functionality.

The loop() function manages the continuous monitoring and control logic. It begins by checking the callHelpFlag variable; if set, a "Call Help" event is sent to the Blynk platform, and the flag is reset. Next, it evaluates the waitSTATE variable to toggle the status LED, indicating whether the patient monitoring system is active.

The function then reads health data from the MAX30100 sensor. If the heart rate exceeds a safe threshold or SpO2 drops below a critical level, the system activates the buzzer and sends an "Emergency" notification to the Blynk platform. Additionally, the loop monitors the HC-SR04 sensor for patient bed occupancy. If the distance exceeds 50 cm, a timer is initiated to prevent false alarms. If the timer expires and the patient has not returned, the system triggers the buzzer and sends an "Alert" notification via Blynk.

Together, the setup() and loop() functions ensure seamless integration of hardware and software components. The system continuously monitors patient health and safety, provides emergency alerts, and facilitates caregiver assistance through both local and remote interactions.

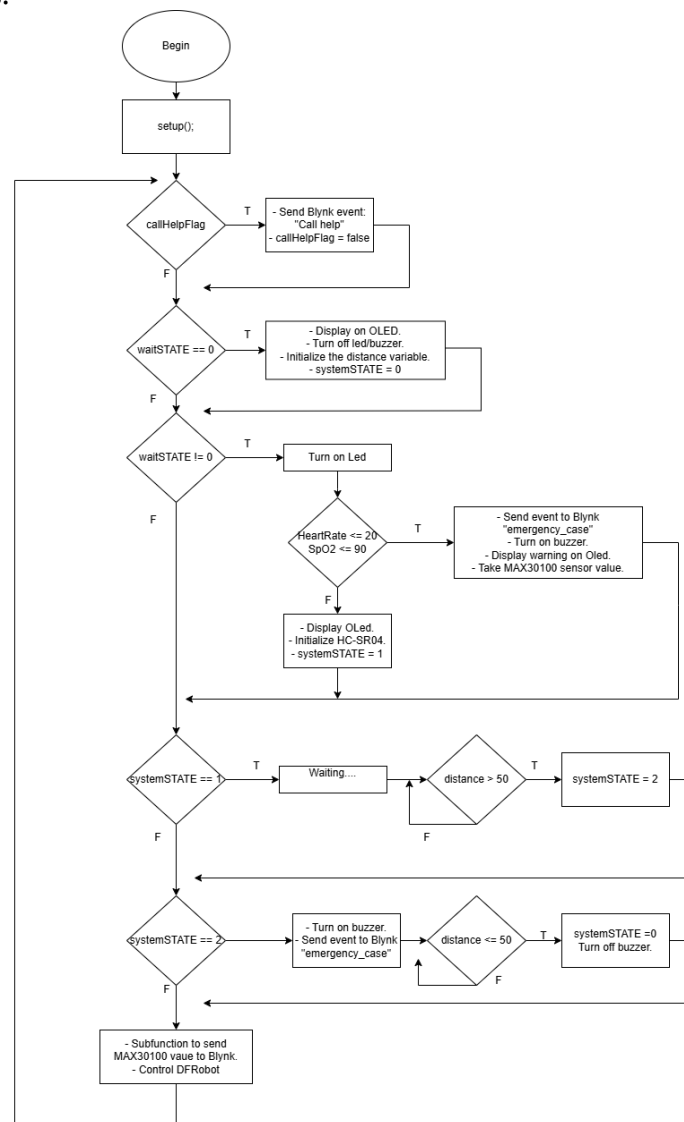


Figure 4.2. Algorithmic flow chart

5. Detailed Design

5.1. Central processing block pin connection

- Library:

```
#include <Wire.h>
#include <ESP32Servo.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SH110X.h>
#include "MAX30100_PulseOximeter.h"
#include "DFRobot_DF2301Q.h"
#include <BlynkSimpleEsp32.h>
#include <WiFi.h>
```

- Pin connection diagram:

Table 5.1. Ultrasonic Sensor HC-SR04 pin connection diagram with ESP32 Node MCU

	HC-SR04	ESP32 Node MCU
1	Echo	G12
2	Trig	G13
3	VCC	5V
4	GND	GND

Table 5.2. Servo SG90 Leg pin connection diagram with ESP32 Node MCU

	Servo SG90 Leg	ESP32 Node MCU
1	Signal	G16
2	VCC	5V
3	GND	GND

Table 5.3. Servo SG90 Head pin connection diagram with ESP32 Node MCU

	Servo SG90 Head	ESP32 Node MCU
1	Signal	G17
2	VCC	5V
3	GND	GND

Table 5.4. OLED pin connection diagram with ESP32 Node MCU

	OLED	ESP32 Node MCU
1	VCC	3.3V
2	GND	GND
3	SCL	G22
4	SDA	G21

Table 5.5. DFRobot pin connection diagram with ESP32 Node MCU

	DFRobot	ESP32 Node MCU
1	VCC	5V
2	GND	GND
3	SCL	G22
4	SDA	G21

Table 5.6. MAX30100 pin connection diagram with ESP32 Node MCU

	MAX30100	ESP32 Node MCU
1	VCC	3.3V
2	GND	GND
3	SCL	G22
4	SDA	G21

Table 5.7. Buttons & Leds & Buzzer component pin connection diagram

		ESP32 Node MCU
1	SSbutton	G34
2	SSled	G5
3	OLbutton	G23
4	Buzzer	G19
5	CHbutton	G26

5.2. Central processing block code

```
#define BLYNK_TEMPLATE_ID "TMPL66Jg8LGVY"
#define BLYNK_TEMPLATE_NAME "SMART HOSPITAL BED"
#define BLYNK_AUTH_TOKEN "fFg70PhjC3S0Ns4n7xzEhiAXbm1KCsqj"
#define BLYNK_PRINT Serial

#include <Wire.h>
#include <ESP32Servo.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SH110X.h>
#include "MAX30100_PulseOximeter.h"
#include "DFRobot_DF2301Q.h"
#include <BlynkSimpleEsp32.h>
#include <WiFi.h>

//////////Ultrasonic Sensor//////////
#define ECHO_PIN 12
#define TRIG_PIN 13

//////////Button&Led&Buzzer//////////
#define SSbutton 34
#define SSled 5
#define OLbutton 23
#define Buzzer 19
#define CHbutton 26

//////////OLED//////////
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define i2c_Address 0x3c

//////////Delay display Serial Monitor//////////
#define REPORTING_PERIOD_MS 1000
```

```
//////////Setting Ultrasonic Sensor Warning//////////
#define THRESHOLD_DISTANCE 50
#define DELAY_TIME 3000

volatile int waitSTATE = 0, displaySTATE = 0;
volatile unsigned long lastInterruptTime = 0;
volatile bool callHelpFlag = false;

Adafruit_SH1106G display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

PulseOximeter pox;
uint32_t tsLastReport = 0;

long duration;
int distance, systemSTATE;

DFRobot_DF2301Q_I2C asr;

Servo sg90_head;
int servoPin_head = 17;
Servo sg90_leg;
int servoPin_leg = 16;

BlynkTimer timer;

const char* auth = BLYNK_AUTH_TOKEN;
const char* ssid = "Vo Ngoc Thanh"; // type your wifi name
const char* pass = "vhk3072003"; // typur wifi password

int blinkOled = 0;
float r;
int p;

static const unsigned char PROGMEM drop_iconHEART[] = {
  B00000000, B00000000,
  B00001110, B00111000,
  B00011111, B01111100,
  B00111111, B11111110,
  B01111111, B11111111,
  B01111110, B00111111,
  B01111110, B00111111,
  B01111000, B00001111,
  B01111000, B00001111,
  B00111000, B00001110,
  B00011110, B00111100,
  B00001110, B00111000,
  B00000111, B11110000,
```

```
B00000011, B11100000,
B00000001, B11000000,
B00000000, B10000000
};

static const unsigned char PROGMEM drop_iconOXY[] = {
    B00000000, B00000110,
    B00000001, B10001001,
    B00000011, B11100010,
    B00001111, B11110111,
    B00011100, B00111000,
    B00011000, B00011000,
    B00110000, B00001100,
    B00110000, B00001100,
    B00110000, B00001100,
    B00110000, B00001100,
    B00011000, B00011000,
    B00011100, B00111000,
    B00001111, B11110000,
    B00000111, B11100000,
    B00000001, B10000000,
    B00000000, B00000000
};

//////////Debounce for Oled display button//////////
void IRAM_ATTR debounce_OLbutton()
{
    unsigned long interruptTime = millis();

    if (interruptTime - lastInterruptTime > 200)
    {
        displaySTATE++;
        if (displaySTATE == 3) displaySTATE = 0;
        lastInterruptTime = interruptTime;
    }
}

//////////Debounce for Ultrasonic Waring button//////////
void IRAM_ATTR debounce_SSbutton()
{
    unsigned long interruptTime = millis();

    if (interruptTime - lastInterruptTime > 200)
    {
        waitSTATE++;
        if (waitSTATE == 2) waitSTATE = 0;
        lastInterruptTime = interruptTime;
    }
}
```

```
    }  
}  
  
//////////Debounce for Calling Help button//////////  
void IRAM_ATTR debounce_CHbutton()  
{  
    unsigned long interruptTime = millis();  
  
    if (interruptTime - lastInterruptTime > 200)  
    {  
        callHelpFlag = true;  
        lastInterruptTime = interruptTime;  
    }  
}  
  
//////////Display on OLED//////////  
void displayOL()  
{  
    pox.update();  
    switch (displaySTATE)  
    {  
        case 1:  
            display.clearDisplay();  
            display.setTextSize(1);  
            display.setTextColor(SH110X_WHITE);  
            display.setCursor(35, 0);  
            display.println("HEART RATE");  
            drawScaledBitmap(15, 15, drop_iconHEART, 16, 16, 2);  
            display.setTextSize(2);  
            display.setCursor(55, 25);  
            display.print(pox.getHeartRate());  
            display.setCursor(40, 45);  
            display.println("(Bpm)");  
            display.display();  
            delay(1);  
            break;  
  
        case 2:  
            display.clearDisplay();  
            display.setTextSize(1);  
            display.setTextColor(SH110X_WHITE);  
            display.setCursor(25, 0);  
            display.println("PULSE OXIMETRY");  
            drawScaledBitmap(20, 15, drop_iconOXY, 16, 16, 2);  
            display.setTextSize(2);  
            display.setCursor(65, 25);  
            display.print(pox.getSpO2());  
            display.setCursor(55, 45);
```

```
        display.println("(%");
        display.display();
        delay(1);
        break;

    default:
        display.clearDisplay();
        display.display();
        delay(1);
        break;
    }
}

//////////Display larger ICON//////////
void drawScaledBitmap(int x, int y, const uint8_t *bitmap, int width, int
height, int scale)
{
    for (int j = 0; j < height; j++)
    {
        for (int i = 0; i < width; i++)
        {
            if (bitmap[j * (width / 8) + i / 8] & (1 << (7 - (i % 8))))
            {
                display.fillRect(x + i * scale, y + j * scale, scale, scale,
SH110X_WHITE);
            }
        }
    }
}

//////////Annouce when have heart beat//////////
void onBeatDetected()
{
    Serial.println("Beat!");
}

//////////Take variable from Ultrasonic Sensor//////////
void UltraSonic_Sensor()
{
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);

    duration = pulseIn(ECHO_PIN, HIGH);
    distance = duration / 58.2;
}
```



```
//////////Turn on/off check if Patient still on bed//////////
void checkPATIENT()
{
    // Check Patient Assistance
    if (callHelpFlag)
    {
        Blynk.logEvent("call_help");
        Serial.println("Goi nguoi ho tro: Success");
        callHelpFlag = false;
    }

    // Check Patient on bed
    if (waitSTATE == 0)
    {
        displayOL();

        digitalWrite(SSled, LOW);
        digitalWrite(Buzzer, LOW);

        distance = 0;
        systemSTATE = 0;
    }
    else
    {
        digitalWrite(SSled, HIGH);

        // Check health patient state
        if(pox.getHeartRate()<=20 || pox.getSpO2()<=90)
        {
            Blynk.logEvent("emergency_case");

            digitalWrite(Buzzer, HIGH);

            //Announcing EMERGENCY SITUATION!!!
            display.clearDisplay();
            display.setCursor(0,0);
            display.setTextSize(1);
            display.setTextColor(SH110X_WHITE);
            display.print("EMERGENCY SITUATION!!");
            display.setCursor(0,15);
            display.setTextColor(SH110X_WHITE);
            display.print("Press button to stop");
            display.display();

            pox.update();
        }
        else
    }
}
```

```
{
    displayOL();
    UltraSonic_Sensor();

    // Check waiting state
    if (systemSTATE == 0)
    {
        systemSTATE = 1;
    }
}

// Waiting process
if (systemSTATE == 1)
{
    unsigned long interruptTime = millis();
    if (interruptTime - lastInterruptTime > 10000)
    {
        lastInterruptTime = interruptTime;
        if (distance > 50)
        {
            systemSTATE = 2;
        }
    }
}

if (systemSTATE == 2)
{
    digitalWrite(Buzzer, HIGH);
    Blynk.logEvent("emergency_case");
    if (distance <= 50)
    {
        systemSTATE = 0; // Quay lại trạng thái tắt
        digitalWrite(Buzzer, LOW);
    }
}
}

//////////Display on Serial Monitor//////////
void displaySM_Control_SendBlynk()
{
    uint8_t CMDID = asr.getCMDID();

    pox.update();
    r = pox.getHeartRate(); //Read the Heart Rate
    p = pox.getSpO2(); // Read the SpO2
```

```
if (millis() - tsLastReport > REPORTING_PERIOD_MS)
{
    Serial.print("Heart rate:");
    Serial.print(pox.getHeartRate());
    Serial.print("bpm / SpO2:");
    Serial.print(pox.getSpO2());
    Serial.println("%");

    Serial.print("Distance: ");
    Serial.print(distance);
    Serial.println(" cm");

    //Blynk set up for Heart Rate and SpO2
    Blynk.virtualWrite(V0, r);
    Blynk.virtualWrite(V1, p);

    tsLastReport = millis();

    Serial.println("-----");
}

if (CMDID != 0)
{
    switch (CMDID)
    {
        //Nang dau giuong
        case
5:

        Serial.println("Nang Dau Giuong: Success");
        sg90_head.write(35);
        break;

        //Ha dau giuong.
        case
6:

        Serial.println("Ha Dau Giuong: Success");
        sg90_head.write(0);
        break;

        //Nang chan
        case
8:

        Serial.println("Nang chan: Success");
        sg90_leg.write(0);
```

```
        break;

        //Ha chan
        case
7:

        Serial.println("Ha chan: Success");
        sg90_leg.write(35);
        break;

        //Tro ve nhu cu
        case
9:

        Serial.println("Tro ve nhu cu: Success");
        sg90_leg.write(0);
        sg90_head.write(0);
        break;

        //Goi nguoi ho tro
        case
10:

        Serial.println("Goi nguoi ho tro: Success");
        Blynk.logEvent("call_help");
        break;

        default:
            Serial.print("CMDID = "); //Printing command ID
            Serial.println(CMDID);
    }
}

}

void setup()
{
    ///Baud rate for serial monitor
    Serial.begin(115200);

    //Oled setup
    display.begin(i2c_Address, true);

    //Display connecting to "WIFI NAME"
    display.clearDisplay();
```

```
display.setCursor(0,0);
display.setTextSize(1);
display.setTextColor(SH110X_WHITE);
display.println("Connecting to");
display.setTextSize(1);
display.setCursor(0,15);
display.print(ssid);
display.display();

//Connect Wifi
WiFi.mode(WIFI_STA);
WiFi.begin(ssid, pass);

//Check if wifi are connected
while (WiFi.status() != WL_CONNECTED)
{
    //Delay to see if Wifi has done connection, if not ...
    delay(500);

    //Blink "..." on OLED
    if (blinkOled == 0)
    {
        display.setTextColor(SH110X_WHITE);
        display.setCursor(80,0);
        display.print("...");
        display.display();

        blinkOled=1;
    }
    else
    {
        display.setTextColor(SH110X_BLACK);
        display.setCursor(80,0);
        display.print("...");
        display.display();

        blinkOled=0;
    }
}

// Connect to Blynk sever
Blynk.begin(auth, ssid, pass);

//Announcing successfully connected to WiFi
display.clearDisplay();
display.setCursor(25,0);
display.setTextSize(1);
display.setTextColor(SH110X_WHITE);
```

```
display.print("WiFi CONNECTED");
display.display();

delay(3000);

///Button&Led&Sensor&Buzzer pins setup
pinMode(CHbutton, INPUT);
pinMode(SSbutton, INPUT);
pinMode(OLbutton, INPUT);
pinMode(SSled, OUTPUT);
pinMode(Buzzer, OUTPUT);

pinMode(TRIG_PIN, OUTPUT);
pinMode(ECHO_PIN, INPUT);

sg90_head.attach(servoPin_head);
sg90_head.write(0);
sg90_leg.attach(servoPin_leg);
sg90_leg.write(0);

attachInterrupt(digitalPinToInterrupt(OLbutton), debounce_OLbutton,
FALLING);
attachInterrupt(digitalPinToInterrupt(SSbutton), debounce_SSbutton,
FALLING);
attachInterrupt(digitalPinToInterrupt(CHbutton), debounce_CHbutton,
FALLING);

///Initialize program
waitSTATE = 0;
displaySTATE = 0;
systemSTATE = 0;

digitalWrite(SSled, LOW);

// Init the DFRobot
while (!(asr.begin()))
{
    Serial.println("Communication with device failed, please check
connection");
    delay(3000);
}
Serial.println("Begin ok!");

asr.setVolume(7);
asr.setMuteMode(0);
asr.setWakeTime(20);

uint8_t wakeTime = 0;
```

```
wakeTime = asr.getWakeTime();
Serial.print("wakeTime = ");
Serial.println(wakeTime);

///Initialize MAX30100
Serial.print("Initializing pulse oximeter..");
if (!pox.begin())
{
    Serial.println("FAILED");
    for (;;)
        ;
}
else
{
    Serial.println("SUCCESS");
}
pox.setOnBeatDetectedCallback(onBeatDetected);
}

void loop()
{
    Blynk.run();
    checkPATIENT();
    displaySM_Control_SendBlynk();
}
```

6. Final Product

6.1. Central processing block

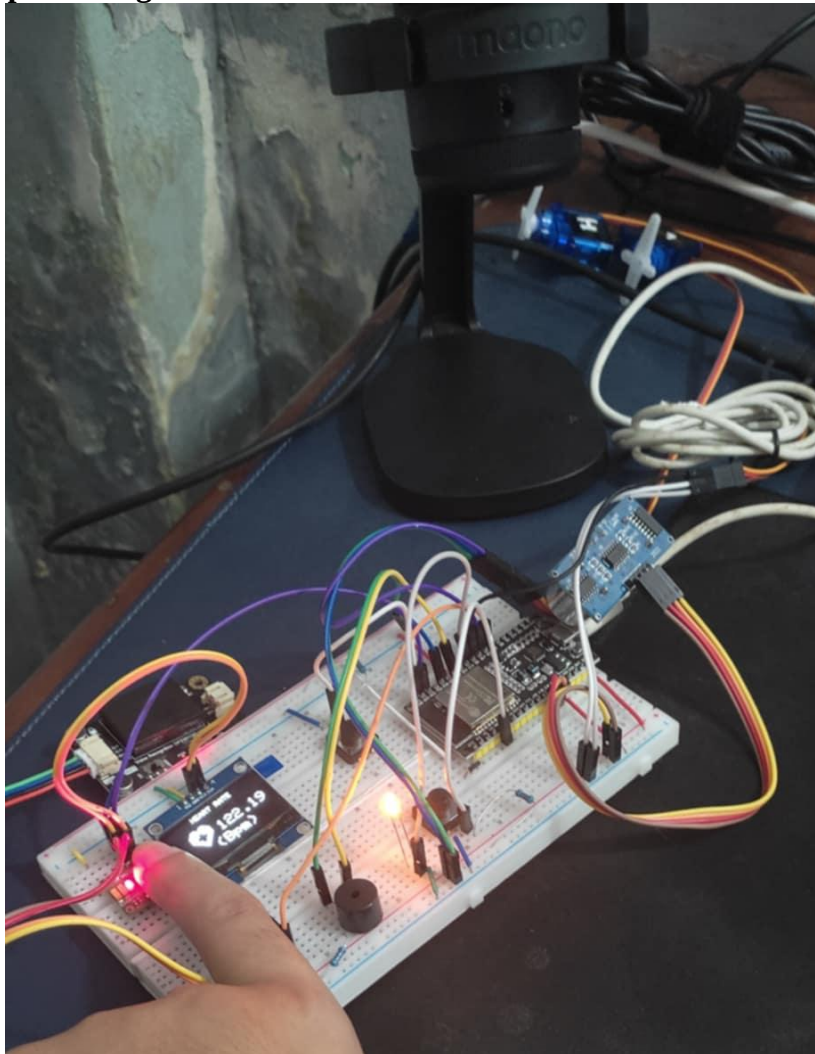


Figure 6.1. Central processing block



Figure 6.2. Final product

6.2. Testing and evaluation:

The system was evaluated based on the performance of its components:

- The MAX30100 sensor provided fluctuating Heart Rate measurements, with BPM readings varying between 100-140 during testing. This indicates some instability in measurement accuracy.
- The HC-SR04 ultrasonic sensor demonstrated quick response times and reliable distance measurements.
- The DFRobot voice recognition module functioned at an acceptable level but occasionally struggled with recognizing Vietnamese voice commands consistently.
- The DFRobot voice module controlling the Servo motor worked reliably. However, the Servo motor rotated quickly, making it less suitable for a hospital bed application. A more appropriate Servo motor should be considered for smoother operation.
- The OLED screen performed well, delivering fast and clear responses when displaying health metrics.
- The Blynk system displayed sensor data almost in real-time, aligning closely with the OLED output. However, notifications and alerts experienced occasional delays in response time.
- Overall, while the system performs its intended functions effectively, improvements in measurement stability, notification delays, and Servo motor selection are recommended for enhanced usability and performance.

6.3. Conclusion and development directions

CONCLUSION

The designed system successfully integrates multiple components to monitor patient health and assist caregivers. It reliably measures Heart Rate and SpO2, displays data on both OLED and Blynk, and provides alert mechanisms for abnormal readings. The system also supports voice commands and manual inputs for additional functionality, such as bed adjustments and calling for assistance. While the system demonstrates good performance overall, some components require refinement to ensure better accuracy, consistency, and user experience.

DEVELOPMENT

Future improvements and expansions for the system include:

- Enhancing the accuracy of the Heart Rate and SpO2 sensor to ensure more reliable health monitoring.
- Replacing the Servo motor with a more suitable motor for smoother and safer bed adjustments.
- Training the DFRobot voice recognition module to support multiple languages and a wider range of commands for increased usability.
- Extending the system to incorporate environmental sensors (e.g., air quality sensors) to monitor the patient's surroundings and enhance their safety.
- Evolving the system from a hospital bed focus to a more comprehensive monitoring solution adaptable to broader healthcare scenarios.

These advancements will not only improve the system's performance but also expand its potential applications in patient care.

7. Appendix

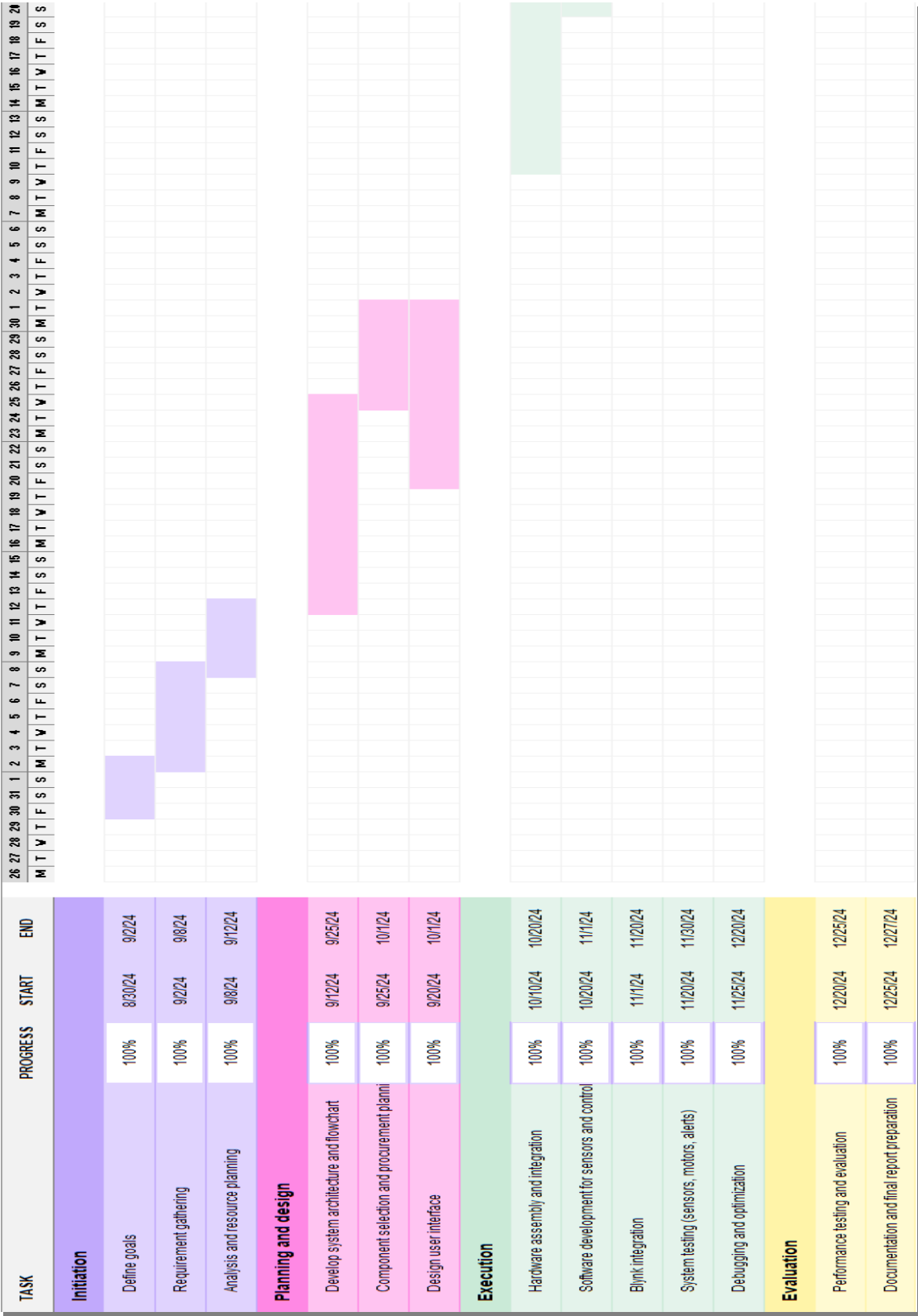
7.1. Division of Labor

TASK	PROGRESS	START	END
Initiation			
Define goals	100%	8/30/24	9/2/24
Requirement gathering	100%	9/2/24	9/8/24
Analysis and resource planning	100%	9/8/24	9/12/24
Planning and design			
Develop system architecture and flowchart	100%	9/12/24	9/25/24
Component selection and procurement planning	100%	9/25/24	10/1/24
Design user interface	100%	9/20/24	10/1/24
Execution			
Hardware assembly and integration	100%	10/10/24	10/20/24
Software development for sensors and controls	100%	10/20/24	11/1/24
Blynk integration	100%	11/1/24	11/20/24
System testing (sensors, motors, alerts)	100%	11/20/24	11/30/24
Debugging and optimization	100%	11/25/24	12/20/24
Evaluation			
Performance testing and evaluation	100%	12/20/24	12/25/24
Documentation and final report preparation	100%	12/25/24	12/27/24

7.2. Bill of Material

ID	Parts/Components	Amount	Price per Unit	Total
1	Servo SG90	2	1.06\$	2.12\$
2	LCD Oled 1.3 inch	1	3.5\$	3.5\$
3	MAX30100 Sensor	1	1.69\$	1.69\$
4	HC-SR04 Sensor	1	0.79\$	0.79\$
5	DFRobot Voice Recognition	1	16.9\$	16.9\$
6	Leds & Buzzer & Button	1	0.79\$	0.79\$
7	Module ESP32	1	5.5\$	5.5\$

7.3. Gantt Chart



7.4. User manual

